

Mastering LangGraph

The Complete 2026 Guide for Python Developers

A Comprehensive Course

2026 Edition

Contents

1	Introduction	2
1.1	Prerequisites	2
1.2	What is LangGraph?	2
1.3	Why LangGraph Exists	2
1.4	LangChain vs LangGraph (2026)	3
2	Core Concepts	4
2.1	The Primitives	4
2.1.1	State	4
2.1.2	Nodes	4
2.1.3	Edges	4
2.1.4	StateGraph, START, and END	4
2.2	Basic Example with Python	4
2.3	Chapter Summary & Exercises	5
3	Agents and Tool Calling	6
3.1	Tool Calling	6
3.2	ReAct Agents	6
3.3	Chapter Summary & Exercises	7
4	Memory and Human-in-the-Loop	8
4.1	Memory & Checkpointing	8
4.2	Human-in-the-Loop (HITL)	8
5	Advanced Orchestration	10
5.1	Streaming	10
5.2	Multi-Agent Systems (2026 Architectures)	10
5.3	Agentic RAG	10
6	Production & Observability	12
6.1	Error Handling	12
6.2	Observability with LangSmith	12
6.3	Deployment (LangSmith Deployment)	12
7	Projects	13
7.1	Project 1: Basic QA Pipeline (Beginner)	13
7.2	Project 2: Stateful Chatbot (Intermediate)	13
7.3	Project 3: HITL Email Assistant (Intermediate-Advanced)	13
7.4	Project 4: Self-Reflective Agentic RAG (Advanced)	13
7.5	Project 5: Production Supervisor Multi-Agent (Expert)	13

Chapter 1

Introduction

1.1 Prerequisites

Definition: The baseline knowledge required to succeed in this course.

Why it is needed: LangGraph is an advanced orchestration framework. You must understand the underlying components before you can orchestrate them.

You should have:

- Intermediate Python (Functions, Classes, Decorators, Async/Await).
- Familiarity with Python type hints and `typing.TypedDict` or `Pydantic`.
- Basic LangChain knowledge (LLMs, Prompts, Tools, simple LCEL chains).
- An API key for an LLM provider (e.g., OpenAI, Anthropic).

1.2 What is LangGraph?

Definition: LangGraph is a stateful, graph-based orchestration runtime for building complex, multi-actor LLM applications.

How it works internally: It represents workflows as directed graphs where nodes are Python functions (agents/tools) and edges define conditional routing. A shared 'State' object flows through the graph, and the graph engine executes nodes and manages persistence via checkpoints.

Real-world analogy: Think of LangChain as the individual workers in an office (a researcher, a writer, a calculator). LangGraph is the *office manager* and the *routing system*. It passes the file (State) between desks (Nodes) based on a set of rules (Edges), and if the power goes out, it remembers exactly whose desk the file was on (Checkpoints).

1.3 Why LangGraph Exists

Why it is needed: Standard LangChain LCEL chains are linear ($A \rightarrow B \rightarrow C$). They cannot handle loops, retries, conditional branching, or pausing for human approval. Real-world AI workflows are non-linear processes. LangGraph was built to provide the underlying engine for agents that need to think, act, observe, and iterate.

1.4 LangChain vs LangGraph (2026)

In October 2025, LangChain and LangGraph released v1.0. They are complementary layers.

- **LangChain** provides the building blocks (LLM wrappers, tools, prompts). It is for pipelines.
- **LangGraph** provides the runtime execution engine. It is for processes.
- **Note:** LangChain's `create_agent` now runs on LangGraph under the hood. You drop down to pure LangGraph when you need explicit control over state, routing, and multi-agent coordination.

Chapter 2

Core Concepts

2.1 The Primitives

LangGraph maps directly to graph theory primitives.

2.1.1 State

Definition: The shared memory object that flows through the graph. **How it works:** Usually defined as a TypedDict or Pydantic model. Each key in the state can have a "reducer" function (like `add_messages`) that determines how updates are merged (e.g., `append` vs. `overwrite`).

2.1.2 Nodes

Definition: Python functions that do the work. **How it works:** A node receives the current State, performs an action (calls an LLM, runs a tool), and returns a dictionary containing updates to the state.

2.1.3 Edges

Definition: The paths between nodes. **How it works:**

- **Normal Edges:** Unconditional transitions (Node A always goes to Node B).
- **Conditional Edges:** A function that evaluates the state and dynamically decides the next node.

2.1.4 StateGraph, START, and END

Definition: `StateGraph` is the builder class. `START` and `END` are special virtual nodes. **How it works:** You add nodes and edges to the `StateGraph`, set an edge from `START` to your first node, and route to `END` when the workflow is finished. Finally, you call `.compile()`.

2.2 Basic Example with Python

Python example: A simple graph that capitalizes text.

```
1 from typing import TypedDict
2 from langgraph.graph import StateGraph, START, END
3
4 # 1. Define State
```

```

5 class GraphState(TypedDict):
6     text: str
7     processed: bool
8
9 # 2. Define Nodes
10 def capitalize_node(state: GraphState) -> GraphState:
11     # Read state
12     current_text = state["text"]
13     # Return updates
14     return {"text": current_text.upper(), "processed": True}
15
16 def add_exclamation_node(state: GraphState) -> GraphState:
17     return {"text": state["text"] + "!!"}
18
19 # 3. Define Conditional Routing
20 def route_next(state: GraphState) -> str:
21     if state["processed"]:
22         return "add_exclamation"
23     return END
24
25 # 4. Build Graph
26 builder = StateGraph(GraphState)
27 builder.add_node("capitalize", capitalize_node)
28 builder.add_node("add_exclamation", add_exclamation_node)
29
30 # 5. Connect Edges
31 builder.add_edge(START, "capitalize")
32 builder.add_conditional_edges("capitalize", route_next)
33 builder.add_edge("add_exclamation", END)
34
35 # 6. Compile and Run
36 app = builder.compile()
37 result = app.invoke({"text": "hello world", "processed": False})
38 print(result) # Output: {'text': 'HELLO WORLD!!', 'processed': True}

```

Line-by-line explanation:

- Line 5-7: We define GraphState. It requires a text string and a processed boolean.
- Line 10-14: capitalize_node takes the state, reads the text, and returns a dictionary with the updated text and boolean flag. LangGraph merges this into the main state.
- Line 20-23: route_next checks the state. If processed is true, it routes to the next node by returning its name string. Otherwise, it returns the special END constant.
- Line 26-36: We assemble the graph, connect START, apply the conditional edge, connect to END, compile, and invoke.

Common mistakes: Returning a string or list from a node instead of a dictionary. Nodes *must* return a dictionary that matches keys in the State schema. **Best practices:** Keep node logic extremely simple. If a node is doing too much, split it into two nodes.

2.3 Chapter Summary & Exercises

Summary: LangGraph workflows consist of a defined State, Python functions acting as Nodes, and conditional logic acting as Edges, all compiled into a StateGraph. **Interview Question:** Explain the difference between LangChain and LangGraph in the context of state management. **Practical Exercise:** Modify the code above to add a third node that reverses the string, and update the routing logic to conditionally send it to the reverse node based on a new boolean flag in the state.

Chapter 3

Agents and Tool Calling

3.1 Tool Calling

Definition: Giving an LLM the ability to trigger external Python functions (tools). **Why it is needed:** LLMs only know text. Tools allow them to interact with the world (search the web, run SQL, fetch APIs). **How it works internally:** You bind tools to an LLM. When prompted, the LLM outputs a specific JSON structure (a `tool_call`) instead of text. LangGraph executes the associated Python function and passes the result back to the LLM.

3.2 ReAct Agents

Definition: Reason + Act. An agent that loops between thinking, acting using tools, observing the result, and thinking again until a final answer is reached. **Real-world analogy:** A detective evaluating a crime scene. They form a hypothesis (Reason), dust for prints (Act), view the results (Observe), and update their hypothesis.

Python example: A basic ReAct loop in LangGraph.

```
1 from typing import Annotated, TypedDict
2 from langchain_openai import ChatOpenAI
3 from langchain_core.tools import tool
4 from langchain_core.messages import BaseMessage, HumanMessage
5 from langgraph.graph import StateGraph, START, END
6 from langgraph.graph.message import add_messages
7 from langgraph.prebuilt import ToolNode, tools_condition
8
9 # 1. Define Tool
10 @tool
11 def get_weather(location: str) -> str:
12     """Gets the weather for a location."""
13     if "london" in location.lower(): return "Rainy, 15C"
14     return "Sunny, 25C"
15
16 tools = [get_weather]
17 llm = ChatOpenAI(model="gpt-4o").bind_tools(tools)
18
19 # 2. Define State (with reducer)
20 class AgentState(TypedDict):
21     # add_messages ensures new messages append, not overwrite
22     messages: Annotated[list[BaseMessage], add_messages]
23
24 # 3. Define Nodes
25 def call_model(state: AgentState):
26     response = llm.invoke(state["messages"])
27     return {"messages": [response]}
```

```

28
29 # Prebuilt ToolNode handles executing the tool calls
30 tool_node = ToolNode(tools)
31
32 # 4. Build Graph
33 graph_builder = StateGraph(AgentState)
34 graph_builder.add_node("agent", call_model)
35 graph_builder.add_node("tools", tool_node)
36
37 graph_builder.add_edge(START, "agent")
38 # Prebuilt conditional edge: goes to tools if tool_call exists, else END
39 graph_builder.add_conditional_edges("agent", tools_condition)
40 graph_builder.add_edge("tools", "agent") # Loop back after tool
41
42 app = graph_builder.compile()
43
44 # Run
45 inputs = {"messages": [HumanMessage(content="What is the weather in London?")]}
46 for chunk in app.stream(inputs, stream_mode="updates"):
47     print(chunk)

```

Line-by-line explanation:

- Line 10-14: We define a standard LangChain tool using the `@tool` decorator.
- Line 20-22: Our state contains a list of messages. We use `Annotated` and `add_messages`. This is a *reducer*. Without it, a node returning `{"messages": [new_msg]}` would overwrite the history. With it, it appends.
- Line 30: We use LangGraph's prebuilt `ToolNode` to automatically execute tools requested by the LLM.
- Line 39-40: We use the prebuilt `tools_condition`. It checks the last message; if it has `tool_calls`, it routes to the "tools" node. Otherwise, it routes to `END`.

Common mistakes: Forgetting the `add_messages` reducer on conversation state. If you forget it, your agent will forget the entire conversation after every node execution. **Best practices:** Use LangGraph's prebuilt modules (like `ToolNode` and `tools_condition`) for standard ReAct loops rather than reinventing them.

3.3 Chapter Summary & Exercises

Summary: LangGraph enables cyclical agent loops. The LLM acts as the routing engine by generating tool calls, which the graph evaluates to dynamically decide whether to execute tools or end the process. **Practical Exercise:** Add a second tool (e.g., a simple math calculator) to the agent above and ask a question requiring both tools.

Chapter 4

Memory and Human-in-the-Loop

4.1 Memory & Checkpointing

Definition: Saving the exact state of the graph at every step to a persistent datastore. **Why it is needed:** For multi-turn conversations (memory), crash recovery, and pausing execution. **How it works internally:** After every node transition, a "checkpointer" serializes the State and saves it alongside a `thread_id`. When the graph is invoked with that `thread_id`, it loads the latest state before starting.

4.2 Human-in-the-Loop (HITL)

Definition: Pausing agent execution to wait for a human to approve, reject, or edit an action. **Why it is needed:** Safety. You do not want an AI agent sending emails, dropping tables, or spending money without human review.

Python example:

```
1 from langgraph.checkpoint.memory import MemorySaver
2 from langgraph.graph import StateGraph, START, END
3
4 # Use a checkpointer
5 memory = MemorySaver()
6
7 # ... (assume graph builder from Chapter 3) ...
8
9 # Compile with checkpointer and set an interrupt BEFORE the tools node
10 app = graph_builder.compile(
11     checkpointer=memory,
12     interrupt_before=["tools"]
13 )
14
15 thread_config = {"configurable": {"thread_id": "user_123"}}
16
17 # 1. Run until interrupt
18 inputs = {"messages": [HumanMessage(content="Send an email to boss@company.com")]}
19 result = app.invoke(inputs, config=thread_config)
20
21 # 2. Graph is paused. Inspect the state.
22 print(app.get_state(thread_config).next) # Outputs: ('tools',)
23
24 # 3. Human approves. Resume execution with None (or a Command to edit)
25 resume_result = app.invoke(None, config=thread_config)
```

Line-by-line explanation:

- Line 10-13: We compile the graph with a checkpointer (`MemorySaver` for local testing) and tell it to `interrupt_before` the "tools" node.
- Line 15: We define a `thread_id`. This is required for checkpoints.
- Line 19: We invoke the app. It will run the "agent" node, see the tool call, route to "tools", hit the interrupt, save the state, and yield back to the script.
- Line 26: We invoke the app again passing `None` as input and the *same* `thread_id`. The graph loads the state, sees it was interrupted at "tools", and resumes execution from that exact spot.

Common mistakes: Trying to use human-in-the-loop without compiling the graph with a checkpointer. HITL fundamentally relies on state persistence. **Production Best Practice:** Use `PostgresSaver` from `langgraph.checkpoint.postgres` in production, not `MemorySaver`.

Chapter 5

Advanced Orchestration

5.1 Streaming

Definition: Receiving data from the graph as it executes, rather than waiting for it to finish. **Why it is needed:** For UI responsiveness. LLMs take time. **How it works:** LangGraph supports multiple stream modes: `values` (emits full state after every node), `updates` (emits only the data returned by the node), and `messages` (streams raw LLM tokens).

5.2 Multi-Agent Systems (2026 Architectures)

Definition: Coordinating multiple specialized LLMs rather than using one giant monolithic prompt. **Why it is needed:** Specialized agents perform better and hallucinate less.

Common Patterns:

1. **Supervisor Pattern:** A router agent evaluates the request and sends it to Worker Agent A or Worker Agent B.
2. **Pipeline/Sequential:** Researcher → Analyst → Writer.
3. **Parallel Fan-Out:** Researcher 1, Researcher 2, and Researcher 3 run simultaneously, and a Reducer agent combines their work.

Python example (Supervisor Routing):

```
1 from typing import Literal
2
3 def supervisor_router(state: AgentState) -> Literal["researcher", "writer", "END"]:
4     # Call an LLM to output a structured decision (researcher, writer, or END)
5     decision = llm_with_structured_output.invoke(state["messages"])
6     return decision.next_node
7
8 # Add conditional edges from Supervisor to workers
9 builder.add_conditional_edges("supervisor", supervisor_router)
10 # Workers always report back to supervisor
11 builder.add_edge("researcher", "supervisor")
12 builder.add_edge("writer", "supervisor")
```

5.3 Agentic RAG

Definition: Retrieval-Augmented Generation controlled by an agent. **Why it is needed:** Standard RAG fails if the query is ambiguous or the retrieval is poor. Agentic RAG evaluates its own

retrieved context. If the context is bad, the agent decides to rewrite the query, search a different database, or search the web.

Key Nodes in Agentic RAG Graph:

- `retrieve_node`: Fetches documents.
- `grade_documents_node`: Uses an LLM to check if documents answer the query.
- `web_search_node`: Fallback if documents fail.
- `generate_node`: Writes final answer.

Chapter 6

Production & Observability

6.1 Error Handling

Definition: Managing API timeouts, LLM structured output failures, and tool crashes. **How it works in LangGraph:** Use `RetryPolicy` attached directly to nodes.

```
1 from langgraph.pregel import RetryPolicy
2
3 retry_policy = RetryPolicy(max_attempts=3, initial_interval=0.5)
4 builder.add_node("brittle_tool", tool_node, retry=retry_policy)
```

6.2 Observability with LangSmith

Definition: Tracing every run, node execution, and LLM call. **Why it is needed:** Debugging agent loops without tracing is impossible. You need to see exactly what tool call the model made that caused a crash. **How to use:** Simply set environment variables: `LANGCHAIN_TRACING_V2=true` and `LANGCHAIN_API_KEY`. LangGraph integrates automatically.

6.3 Deployment (LangSmith Deployment)

In 2026, the standard deployment path for LangGraph is LangSmith Deployment. It provides native support for long-running workflows, large state persistence in Postgres, and webhook handling. For self-hosting, applications are typically wrapped in FastAPI, using the `langgraph_sdk`.

Chapter 7

Projects

7.1 Project 1: Basic QA Pipeline (Beginner)

Goal: Create a simple sequential graph (Linear, no agents). **Structure:** State contains query and answer. Two nodes: retriever and generator. No conditional edges.

7.2 Project 2: Stateful Chatbot (Intermediate)

Goal: Build a conversational bot with memory. **Structure:** Use `add_messages` reducer in state. Add `MemorySaver`. Test resuming conversations using `thread_id`.

7.3 Project 3: HITL Email Assistant (Intermediate-Advanced)

Goal: An agent that drafts emails but requires user approval. **Structure:** Graph with Agent Node and Tool Node. Set `interrupt_before=["tools"]`. Implement the logic to pause, print the drafted email, accept user input via `input()`, and resume the graph with `Command(resume=input_data)`.

7.4 Project 4: Self-Reflective Agentic RAG (Advanced)

Goal: RAG that grades its own retrieval. **Structure:** Implement nodes for retrieval, grading, generating, and rewriting. **Routing Logic:** *Retrieve → Grade. If Grade is pass → Generate. If Grade is fail → Rewrite Query → Retrieve.*

7.5 Project 5: Production Supervisor Multi-Agent (Expert)

Goal: Build a research team. **Structure:**

- State: Complex TypedDict tracking messages, draft_report, research_notes.
- Nodes: Supervisor, Web Searcher, Data Analyst, Editor.
- Persistence: Use PostgresSaver.
- Error Handling: Implement RetryPolicy on all tool nodes.